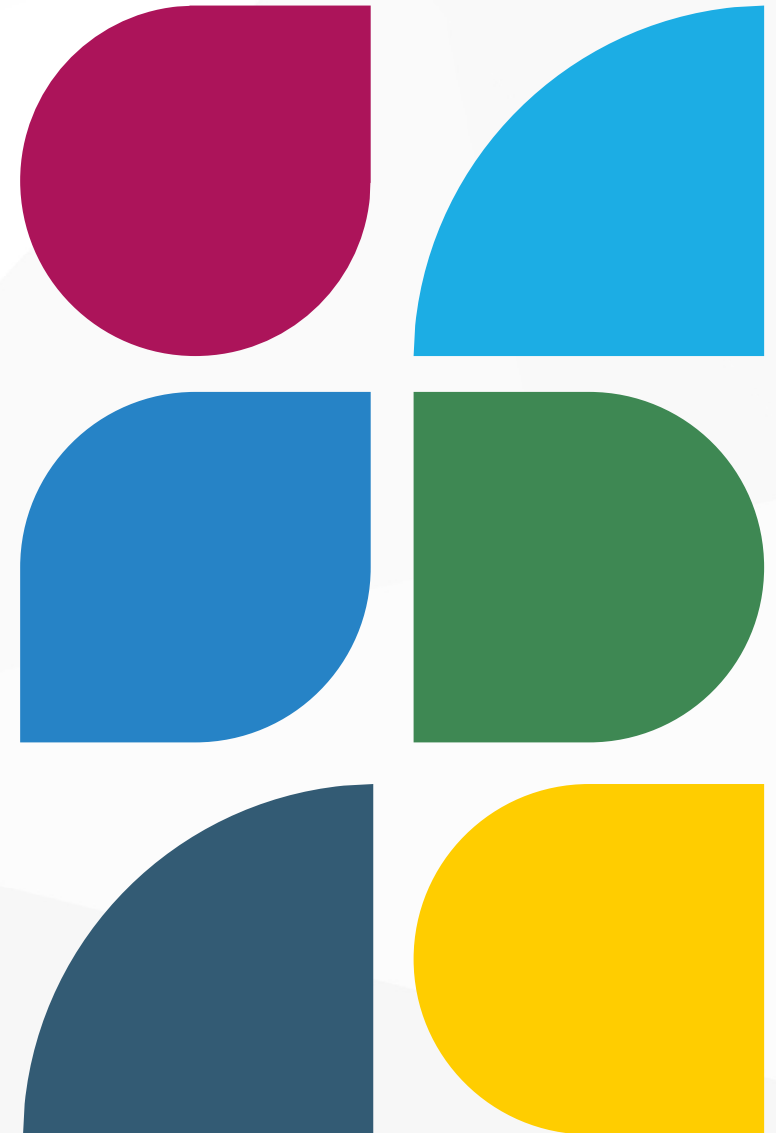


LE LANGAGE SQL STRUCTURED QUERY LANGUAGE

Fonctionnalités complémentaires de SQL : REGEX, Gestion des erreurs, les triggers et contrôle des accès.





MYSQL - REGEX

UTILISATION DES REGEX AVEC SQL

UTILISATION DES REGEX

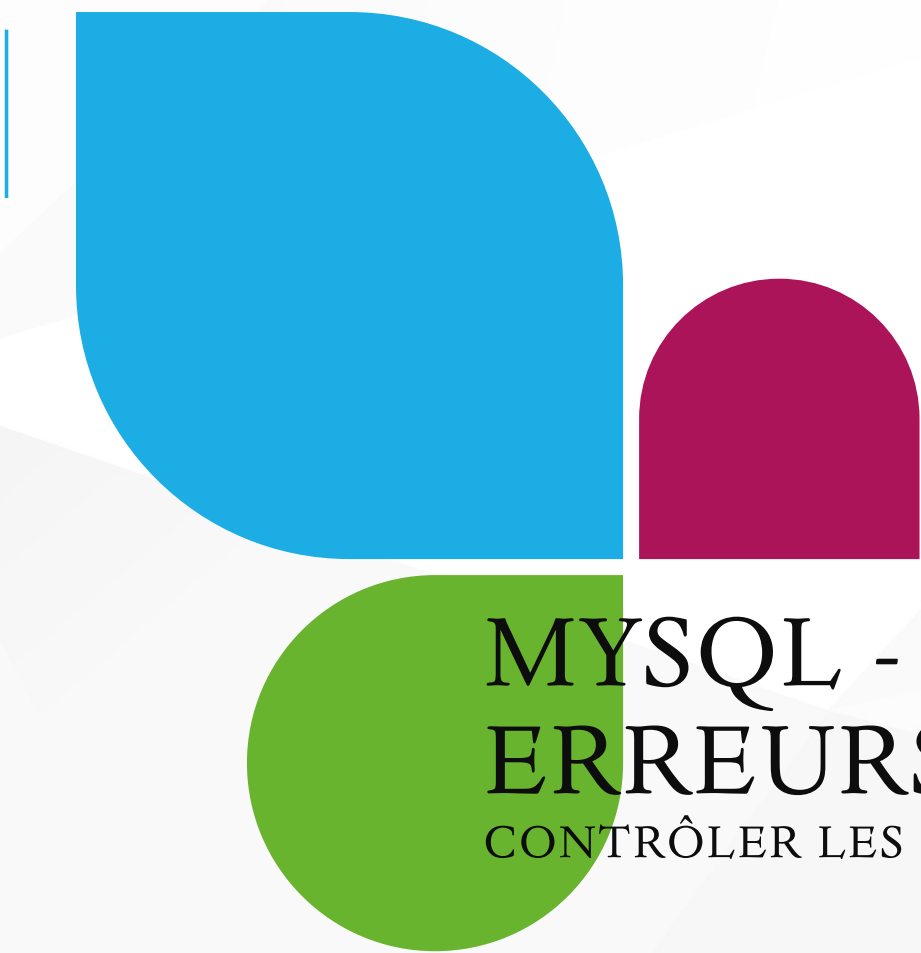
MySQL utilise l'opérateur REGEXP ou RLIKE pour la validation d'une chaîne de caractères :

```
-- Syntaxe
SELECT 'string' REGEXP 'pattern'

-- exemples
SELECT 'a' REGEXP '^[a-z]'; -- 1
SELECT 'A' REGEXP '^[a-z]'; -- 0
SELECT '1' REGEXP '^[a-z]'; -- 0
SELECT 'é' REGEXP '^[a-z]'; -- 0

-- donne les noms commençant par 'sa'
SELECT name FROM student_tbl WHERE name REGEXP '^sa';
-- donne les noms terminant par 'on'
SELECT name FROM student_tbl WHERE name REGEXP 'on$';
-- donne les noms contenant une lettre comprise entre 'b' et 'g'
-- suivi de n'importe quel caractère, suivi de la lettre 'a'
SELECT name FROM student_tbl WHERE name REGEXP '[b-g].[a]';

select REGEXP_REPLACE('hello+$û^:world', '^[a-z]', '');
-- helloworld
```



MYSQL - GESTION DES ERREURS

CONTRÔLER LES ERREURS

SIGNAL ET RESIGNAL

Avec MySQL, on génère l'exception à l'aide de l'instruction `SIGNAL`.

- *`RESIGNAL` permet de modifier une erreur existante avant de la propager.*

Pour une exception personnalisée, il est de coutume d'utiliser le `SQLSTATE '45000'`.

On peut ensuite donner au `SIGNAL` un "numéro d'erreur" `MYSQL_ERRNO` qui doit être un nombre à 5 chiffres.

- *Il vaut mieux éviter d'utiliser les numéros standards de MySQL et donc commencer la numérotation des exceptions personnalisées à partir de 10001.*

Enfin, Le message personnalisé est donné par la variable `MESSAGE_TEXT` et ne doit pas dépasser 64 caractères sous peine d'être tronqué.

```
DELIMITER //
CREATE PROCEDURE valider_patient(IN id_patient INT)
BEGIN
    DECLARE patient_existe INT;

    SELECT COUNT(*) INTO patient_existe
    FROM patients
    WHERE id = id_patient;

    IF patient_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le patient avec cet ID n'existe pas.';
    END IF;

    -- Logique si le patient existe
    SELECT * FROM patients WHERE id = id_patient;
END //
DELIMITER ;
```

DÉFINIR UN GESTIONNAIRE D'EXCEPTION

DECLARE EXIT HANDLER FOR SQLEXCEPTION

Un handler est une sorte de gestionnaire d'erreur (ou de condition).

Cela permet de gérer les exceptions avec DECLARE HANDLER. C'est utilisé pour définir dans une procédure stockée, une fonction ou un déclencheur qui permet d'interrompre le code en cas d'erreurs.

Elle permet de capturer et de traiter les erreurs SQL qui surviennent pendant l'exécution du bloc de code.

- **EXIT** : Indique que le bloc doit être quitté après l'exécution du gestionnaire.
- **SQLEXCEPTION** : Capture toutes les erreurs SQL
 - Il existe aussi **SQLWARNING** pour les avertissements.

Exemple

```
DELIMITER //
```

```
CREATE PROCEDURE ajouter_patient(IN nom VARCHAR(100), IN prenom VARCHAR(100))
```

```
BEGIN
```

```
    -- Déclarer un gestionnaire d'exception
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        -- En cas d'erreur, afficher un message personnalisé
```

```
        SELECT 'Erreur : Impossible d'ajouter le patient.' AS message;
```

```
        -- Optionnel : journaliser l'erreur dans une table
```

```
        INSERT INTO logs_erreurs (message, date_erreur)
```

```
        VALUES (CONCAT('Erreur lors de l'ajout du patient : ', MESSAGE_TEXT), NOW());
```

```
    END;
```

```
    -- Logique principale
```

```
    INSERT INTO patients (nom, prenom) VALUES (nom, prenom);
```

```
    SELECT 'Patient ajouté avec succès.' AS message;
```

```
END //
```

```
DELIMITER ;
```



MYSQL - LES TRIGGERS

METTRE EN PLACE DES DÉCLENCHEURS

LES TRIGGERS

Définition

Le trigger

- comme une procédure mais exécutée automatiquement.
- créés pour effectuer certaines tâches en réponse à l'occurrence d'un événement spécifié (**INSERT**, **DELETE** et **UPDATE**).

À quoi sert un trigger ?

- Contraintes et vérifications de données
- Intégrité des données
- Historisation des actions
- Mise à jour d'informations qui dépendent d'autres données

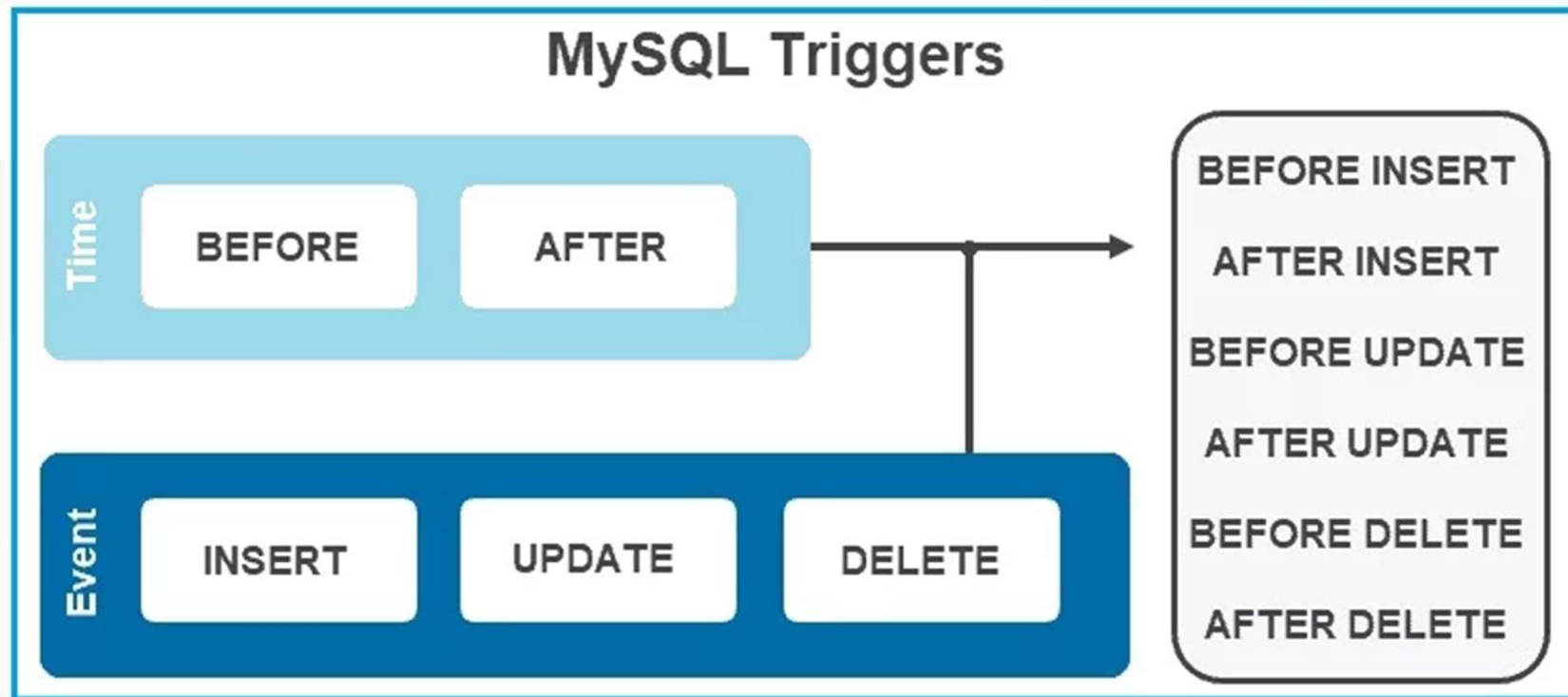
Principe

Lorsqu'un trigger est déclenché, ses instructions peuvent être exécutées à deux moments différents : soit juste avant que l'événement déclencheur n'ait lieu (**BEFORE**), soit juste après (**AFTER**).

Il ne peut exister qu'un seul trigger par combinaison [moment du trigger/événement du trigger] par table.

On a donc **un maximum de 6 triggers par table**.

RAPPEL



LES TRIGGERS

Syntaxe

```
-- syntaxe du trigger
CREATE TRIGGER nom_du_trigger
BEFORE/AFTER <events>
| CONDITION
ACTION;

-- exemple
CREATE OR REPLACE TRIGGER employee_commission_trigger
| BEFORE INSERT ON employee
| FOR EACH ROW
BEGIN
| IF NEW.deptno = 22 THEN
|   NEW.commission := NEW.salary * .2;
| END IF;
END;

DROP TRIGGER nom_du_trigger;
```

Les mots-clés OLD et NEW

Dans le corps du trigger, MySQL met à disposition deux mots-clés : **OLD** et **NEW**.

OLD représente les valeurs des colonnes de la ligne traitée avant qu'elle ne soit modifiée par l'événement déclencheur. Ces valeurs peuvent être lues, mais pas modifiées.

NEW représente les valeurs des colonnes de la ligne traitée après qu'elle a été modifiée par l'événement déclencheur. Ces valeurs peuvent être lues et modifiées.

EXEMPLE

Pour un UPDATE :

- OLD et NEW existeront

Pour un INSERT :

- OLD n'existe pas
- NEW existera

Pour un DELETE :

- OLD existera
- NEW n'existera pas

```
delimiter |
create trigger before_update_contract before update
on CONTRACT for each row
begin
    -- cas du CDD
    if old.idStatus = '2' and new.idStatus > 2 then
        signal sqlstate "45000" set message_text = "Modification du status CDD en erreur";
    -- cas du CDI
    elseif old.idStatus = '1' and new.idStatus > 1 then
        signal sqlstate "45000" set message_text = "Modification du status CDI en erreur";
    -- cas du STA
    elseif old.idStatus = '3' and new.idStatus != '3' then
        signal sqlstate "45000" set message_text = "Modification du status STA en erreur";
    end if;
end |
```



MYSQL - GESTION DES UTILISATEURS

GESTIONS DES ACCÈS AUX DONNÉES

LES UTILISATEURS ET LEURS PRIVILÈGES

Privilèges

Lorsqu'on se connecte à MySQL, on le fait avec un utilisateur possédant des privilèges :

- **Privilèges globaux** : Appliqués à toutes les bases de données (ex: CREATE USER, SHUTDOWN).
- **Privilèges au niveau de la base de données** : Appliqués à une base spécifique (ex: SELECT, INSERT, DELETE).
- **Privilèges au niveau des tables/colonnes** : Appliqués à des objets spécifiques (ex: UPDATE sur une colonne précise).
- **Privilèges administratifs** : Gestion des utilisateurs et des rôles (ex: GRANT OPTION).

Stockage des utilisateurs et privilèges

- Toutes les informations de gestion des utilisateurs et de leurs privilèges sont stockées dans la base de données système **mysql**.
- Les utilisateurs et leurs privilèges globaux (appliqués à tout le serveur) sont définis dans la table **user**.
- MySQL utilise quatre tables supplémentaires pour gérer les privilèges à des niveaux plus fins :
 - **db** : Privilèges au niveau des bases de données (ex: SELECT, INSERT).
 - **tables_priv** : Privilèges au niveau des tables (ex: UPDATE, DELETE).
 - **columns_priv** : Privilèges au niveau des colonnes (ex: accès à une colonne spécifique).
 - **proc_priv** : Privilèges au niveau des procédures et fonctions stockées (ex: droit d'exécution).

LES PRIVILEGES

Il existe de nombreux privilèges dont voici une sélection des plus utilisés.

- Les privilèges SELECT, INSERT, UPDATE et DELETE permettent aux utilisateurs d'utiliser ces mêmes commandes.
- Les privilèges concernant les tables, les bases de données (création, modification etc..)
- D'autres privilèges sur les verrous, les triggers, les procédures stockées etc...

```
GRANT privilege [(liste_colonnes)] [, privilege [(liste_colonnes)], ...]
ON [type_objet] niveau_privilege
TO utilisateur [IDENTIFIED BY mot_de_passe];

-- exemple
GRANT SELECT,
    UPDATE (nom, sexe, commentaires),
    DELETE,
    INSERT
ON elevage.Animal
TO 'john'@'localhost' IDENTIFIED BY 'exemple2012';

REVOKE privilege [, privilege, ...]
ON niveau_privilege
FROM utilisateur;

--exemple
REVOKE DELETE
ON elevage.Animal
FROM 'john'@'localhost';
```

LES DIFFÉRENTS NIVEAUX D'APPLICATION

Niveau	Application du privilège
.	Privilège global : s'applique à toutes les bases de données, à tous les objets. Un privilège de ce niveau sera stocké dans la table mysql.user.
*	Si aucune base de données n'a été préalablement sélectionnée (avec USE nom_bdd), c'est l'équivalent de . (privilège stocké dans mysql.user). Sinon, le privilège s'appliquera à tous les objets de la base de données qu'on utilise (et sera stocké dans la table mysql.db).
nom_bdd.*	Privilège de base de données : s'applique à tous les objets de la base nom_bdd (stocké dans mysql.db).
nom_bdd.nom_table	Privilège de table (stocké dans mysql.tables_priv).
nom_table	Privilège de table : s'applique à la table nom_table de la base de données dans laquelle on se trouve, sélectionnée au préalable avec USE nom_bdd (stocké dans mysql.tables_priv).
nom_bdd.nom_routine	S'applique à la procédure (ou fonction) stockée nom_bdd.nom_routine (privilège stocké dans mysql.procs_priv).

PRIVILÈGES PARTICULIERS

ALL et USAGE

```
-- ACCORDER LES PRIVILEGES
GRANT ALL
ON bdd.nom_Table
TO user;

-- N'ACCORDER AUCUN PRIVILEGES
GRANT USAGE
ON bdd.nom_Table
TO user;

-- exemple
GRANT ALL
ON sparadrap.*
TO 'jerome'@'localhost' IDENTIFIED BY 'ceciestunmotdepasse';
```

GRANT OPTION

Un utilisateur ayant ce privilège est autorisé à utiliser la commande GRANT, pour accorder des privilèges à d'autres utilisateurs.

On peut accorder GRANT OPTION de deux manières :

- comme un privilège normal, après le mot GRANT
- à la fin de la commande GRANT, avec la clause WITH GRANT OPTION.

```
GRANT SELECT, UPDATE, INSERT, DELETE
ON sparadrap.*
TO 'jerome'@'localhost' IDENTIFIED BY 'ceciestunmotdepasse'
WITH GRANT OPTION;
```


PRIVILÈGES PARTICULIERS

Définisseur

Les procédures stockées, les triggers, les vues etc.. ont les privilèges de l'utilisateur qui a créé ces objets.

Donc si on ne souhaite pas qu'un utilisateur n'ayant pas accès à une table visée par une procédure, il faut penser à l'indiquer lors de la définition de l'objet.

```
-- Procédures
CREATE
  [DEFINER = { utilisateur | CURRENT_USER }]
  PROCEDURE nom_procedure ([parametres_procedure])
  SQL SECURITY { DEFINER | INVOKER }
  corps_procedure
```

Limitation des ressources

On peut limiter trois choses différentes pour un utilisateur :

- le nombre de requêtes par heure (MAX_QUERIES_PER_HOUR) : limitation de toutes les commandes exécutées par l'utilisateur
- le nombre de modifications par heure (MAX_UPDATES_PER_HOUR) : limitation des commandes entraînant la modification d'une table ou d'une base de données
- le nombre de connexions au serveur par heure (MAX_CONNECTIONS_PER_HOUR).

```
GRANT ALL ON elevege.*
TO 'aline'@'localhost' IDENTIFIED BY 'limited'
WITH MAX_QUERIES_PER_HOUR 50
     MAX_CONNECTIONS_PER_HOUR 5;
```

FLUSH PRIVILEGES

La commande `FLUSH PRIVILEGES` permet de recharger les tables contenant les différents privilèges sur le serveur.

<https://dev.mysql.com/doc/refman/5.7/en/privilege-changes.html>

Cependant, certaines modifications de privilèges n'ont pas besoin d'être rechargées.

1. Si vous modifiez des privilèges sur des tables, ces modifications ne seront pas prises en compte immédiatement.
 - Dans ce cas, il faut effectuer un `FLUSH PRIVILEGES` afin d'actualiser les tables concernées. Sinon, il faudra attendre un redémarrage du serveur pour la bonne prise en compte des modifications.
2. Si vous utilisez l'une des commandes suivantes : `GRANT`, `SET PASSWORD` ou `RENAME USER` alors le serveur applique immédiatement les modifications et dans ce cas pas besoin d'utiliser `FLUSH PRIVILEGES`.

GESTION DES UTILISATEURS

L'utilisateur est donc défini par deux éléments :

- son login
- l'hôte à partir duquel il se connecte.
- son mot de passe

Notes : Dans [MySQL workbench](#), vous pouvez directement créer les comptes utilisateur et gérer leurs privilèges

- Création et suppression

```
-- CREATION
CREATE USER 'login'@'hote' [IDENTIFIED BY 'password'];

-- SUPPRESSION
DROP USER IF EXISTS 'login'@'hote';
```

- Renommer l'utilisateur et mot de passe

```
-- modifier l'identifiant d'un utilisateur
RENAME USER 'jero'@'localhost' TO 'jerome'@'localhost';

-- modifier le mot de passe de l'utilisateur
SET PASSWORD FOR 'jerome'@'localhost' = PASSWORD('ceciestunmotdepasse');
```

MYSQL WORKBENCH

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard

Administration

Information



MYSQL

Users and Privileges

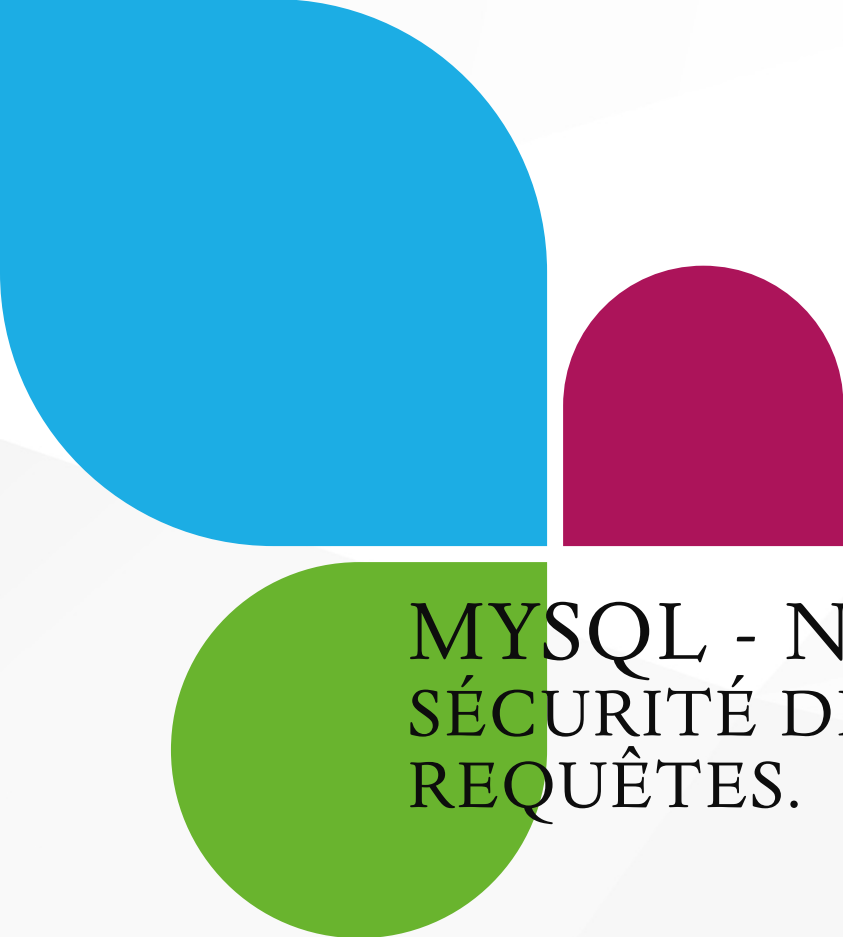
User Accounts

User	From Host
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
root	localhost

Details for account root@localhost

[Login](#) [Account Limits](#) [Administrative Roles](#) [Schema Privileges](#)

Role	Description
☒ DBA	grants the rights to perform all tasks
☒ MaintenanceAdmin	grants rights needed to maintain server
☒ ProcessAdmin	rights needed to assess, monitor, and kill any user proce...
☒ UserAdmin	grants rights to create users logins and reset passwords
☒ SecurityAdmin	rights to manage logins and grant and revoke server an...
☒ MonitorAdmin	minimum set of rights needed to monitor server
☒ DBManager	grants full rights on all databases
☒ DBDesigner	rights to create and reverse engineer any database sche...
☒ ReplicationAdmin	rights needed to setup and manage replication
☒ BackupAdmin	minimal rights needed to backup any database
☒ Custom	custom role



MYSQL - NOTION DE SÉCURITÉ SÉCURITÉ DE NOS ACCÈS ET DE NOS REQUÊTES.

SÉCURITÉ

Principe

Une des attaques courantes aujourd'hui est l'injection de code généralement par le biais d'un formulaire web ou statique.

Les BDD stockent les données en liaison avec nos formulaires.

Par conséquent, il faut donc examiner quelques notions de sécurité en matière de SQL.

Paramètres de connexion

Également, il est important de bien choisir le mot de passe d'accès à sa base de données et de le stocker dans les fichiers de configuration à l'abri des regards indiscrets.

*Note : dans votre apprentissage, on a choisi root / root comme login et mot de passe. Évidemment; dans la pratique, **c'est à bannir**.*

***ET** Il ne faudra surtout pas garder ce genre de mot de passe dans la base de données ...*

IL EXISTE UN ADAGE BIEN CONNU EN PROGRAMMATION : "NEVER TRUST USER INPUT" TRADUIT EN FRANÇAIS PAR "NE JAMAIS FAIRE CONFIANCE AUX DONNÉES FOURNIES PAR L'UTILISATEUR".

MAIS QU'EST-CE QU'UNE **INJECTION SQL** ?

ON APPELLE INJECTION SQL LE FAIT QU'UN UTILISATEUR FOURNISSE DES DONNÉES CONTENANT DES MOTS-CLÉS SQL OU DES CARACTÈRES PARTICULIERS QUI VONT DÉTOURNER OU MODIFIER LE COMPORTEMENT DES REQUÊTES CONSTRUITES SUR LA BASE DE CES DONNÉES.

Formulaire de login

Connexion

Nom d'utilisateur

Mot de passe

LOGIN

en JAVA par exemple

Ici, dans notre code, on a codé la requête en concaténant la saisie

```
private void controleSaisie(String name, String password) { no usages

    String query = "SELECT * FROM users WHERE username = '"
        + name
        + "' AND password = '"
        + password
        + "';"

    // connexion à la BDD et Envoi de la requete...
```

L'ATTAQUE

Désormais, votre page de connexion est fonctionnelle et un attaquant tente de tester la page par différentes saisies malveillantes :

1. `admin'; --'` et `password`
2. `toto' OR '1'='1` et `toto'OR '1'='1`

Que se passe-t-il au niveau applicatif puis au niveau SQL ?



- Cas 1 ?

Au niveau Java, nous complétons notre requête avec la saisie qui donnera :

```
SELECT * FROM users WHERE username = 'admin'; -- "  
AND password = 'password';
```

- Cas 2 ?

Au niveau Java, nous complétons notre requête avec la saisie qui donnera :

```
SELECT * FROM users WHERE username = 'toto' OR '1' =  
'1' AND password = 'toto' OR '1' = '1';
```

Et au niveau SQL ?

```
SELECT * FROM users WHERE username = 'admin'; -- ' AND password = 'toto';
```

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'toto' OR '1'='1';
```


LA SOLUTION

Pour éviter ce type d'attaque, nous allons tout simplement éviter la concaténation **qui est à bannir** !!!

Nous allons envoyer la requête au serveur en lui indiquant que celle-ci contient des paramètres dans un type attendu.

Le serveur va stocker la requête en mémoire dans l'attente des paramètres

Dans un 2^{ème} temps, nous transmettons les paramètres au serveur qui va alors compléter la requête.

```
private void controleSaisie(String name, String password) { 1 usage

    String query = "SELECT * FROM users WHERE username = ? AND password = ?";

    // connexion à la BDD et Envoi de la requete...

    // envoi des paramètres
    query.setString(1, name);
    query.setString(2, password);

    // exécution
```

Désormais, MySQL s'attend à un type de donnée. Tout d'abord JAVA protège les paramètres par son typage fort.. Il veut un String et rien d'autres.

Ici des chaînes de caractères donc **admin'; --** ' sera considéré en un seul bloc et non comme une concaténation. Ma requête SQL deviendra :

```
SELECT * FROM users WHERE username = 'admin'; -- " ...
```

Dans ma base, je n'ai aucun utilisateur nommé **admin'; --** '

EVITER LES INJECTIONS

En utilisant les requêtes préparées, lorsque vous liez une valeur à un paramètre de la requête préparée grâce à la fonction correspondante dans le langage que vous utilisez, le type du paramètre attendu est également donné, explicitement ou implicitement.

La plupart du temps, soit une erreur sera générée si la donnée de l'utilisateur ne correspond pas au type attendu, soit la donnée de l'utilisateur sera rendue inoffensive (par l'ajout de guillemets qui en feront une simple chaîne de caractères par exemple).

Par ailleurs, lorsque MySQL injecte les valeurs dans la requête, les mots-clés SQL qui s'y trouveraient pour une raison ou une autre, ne seront pas interprétés (puisque les paramètres n'acceptent que des valeurs, et pas des morceaux de requêtes).

Voici les étapes lorsqu'il s'agit d'une requête préparée :

1. Préparation de la requête :

1. La requête est envoyée vers le serveur, avec un identifiant.
2. La requête est compilée par le serveur.
3. Le serveur crée un plan d'exécution de la requête.

2. La requête compilée et son plan d'exécution sont stockés en mémoire par le serveur.

3. Le serveur envoie vers le client une confirmation que la requête est prête (en se servant de l'identifiant de la requête).

4. Exécution de la requête :

1. L'identifiant de la requête à exécuter, ainsi que les paramètres à utiliser sont envoyés vers le serveur.
2. Le serveur exécute la requête demandée.

5. Le résultat de la requête est renvoyé au client.

LES REQUÊTES PRÉPARÉES EN SQL

Préparation d'une requête

Pour préparer une requête, il faut renseigner deux éléments :

- le nom qu'on donne à la requête préparée
- la requête elle-même, avec un ou plusieurs paramètres (**représentés par un ?**).

Important :

- Le nom de la requête ne doit pas être en guillemets.
- La requête doit être passée comme une chaîne de caractères et ne doit contenir qu'une seule requête.
- Les paramètres représentent des données, valeurs mais pas des noms de tables ou de colonnes.

Exemples

```
-- Sans paramètre
PREPARE select_race
FROM 'SELECT * FROM Race';

-- Avec un paramètre
PREPARE select_client
FROM 'SELECT * FROM Client WHERE email = ?';

-- Avec deux paramètres
PREPARE select_adoption
FROM 'SELECT * FROM Adoption WHERE client_id = ? AND animal_id = ?';

-- exemple
SET @req = 'SELECT * FROM Race';
PREPARE select_race
FROM @req;

SET @colonne = 'nom';
SET @req_animal = CONCAT('SELECT ', @colonne, ' FROM Animal WHERE id = ?');
PREPARE select_col_animal
FROM @req_animal;
```

LES REQUÊTES PRÉPARÉES

Exécution et suppression

Pour exécuter une requête préparée, on utilise la commande suivante :

```
EXECUTE nom_requete [USING @parametre1, @parametre2, ...];
```

Pour supprimer une requête préparée, on utilise **DEALLOCATE PREPARE**, suivi du nom de la requête préparée :

```
DEALLOCATE PREPARE select_race;
```

Quelques exemples

```
-- exemples
EXECUTE select_race;

SET @id = 3;
EXECUTE select_col_animal USING @id;

SET @client = 2;
EXECUTE select_adoption USING @client, @id;

SET @email = 'jean.dupont@email.com';
EXECUTE select_client USING @email;

SET @email = 'marie.boudun@email.com';
EXECUTE select_client USING @email;

SET @email = 'fleurtrachon@email.com';
EXECUTE select_client USING @email;
```

USAGE DES REQUÊTES PRÉPARÉES



Usage

La syntaxe vue est très rarement utilisée.

En effet, vous le savez, MySQL est rarement utilisé seul. La plupart du temps, il est utilisé en conjonction avec un langage de programmation, comme Java, PHP, python, etc.

Un langage comme JAVA utilise alors une API (interface de programmation) pour faire des requêtes préparées sans exécuter vous-mêmes les commandes SQL.

Nous verrons cela prochainement.

Pourquoi ?

Les requêtes préparées sont principalement utilisées pour deux raisons :

- protéger son application des injections SQL
- gagner en performance dans le cas d'une requête exécutée plusieurs fois par la même session.